3364. Minimum Positive Sum Subarray

Daily Question - November 14, 2026

Problem Statement

You are given an integer array nums and two integers l and r. Find the minimum sum of a subarray whose size is between l and r, inclusive, and whose sum is greater than 0.

Return the minimum sum of such a subarray. If no such subarray exists, return -1. A subarray is a contiguous non-empty sequence of elements within an array.

Examples

Example 1: nums = [3,-2,1,4], l = 2, r = 3. Output: 1. The subarray [3,-2] has length 2 and sum 1, which is the minimum positive sum.

Example 2: nums = [-2,2,-3,1], l = 2, r = 3. Output: -1. No subarray of length between 2 and 3 has a positive sum.

Example 3: nums = [1,2,3,4], l = 2, r = 4. Output: 3. The subarray [1,2] has length 2 and sum 3.

Constraints

1 <= nums.length <= 100; 1 <= l <= r <= nums.length; -1000 <= nums[i] <= 1000.

Solution: Sliding Window for Each Length

For each allowed subarray length size from l to r, maintain a sliding window of exactly that length. Compute the sum of the first window, then slide it across the array by adding the entering element and removing the leaving element.

Update the answer whenever the current window sum is strictly greater than 0. If no valid positive sum is found, return -1.

Complete Java Solution

```
import java.util.List;

class Solution {
    public int minimumSumSubarray(List<Integer> nums, int l, int r) {
        int n = nums.size();
        int answer = Integer.MAX_VALUE;

        for (int size = l; size <= r; size++) {
            int windowSum = 0;

            for (int index = 0; index < size; index++) {
                windowSum += nums.get(index);
            }

            if (windowSum > 0) {
```

```
                answer = Math.min(answer, windowSum);
            }

            for (int end = size; end < n; end++) {
                windowSum += nums.get(end);
                windowSum -= nums.get(end - size);

                if (windowSum > 0) {
                    answer = Math.min(answer, windowSum);
                }
            }
        }

        return answer == Integer.MAX_VALUE ? -1 : answer;
    }
}
```

Correctness

For a fixed length size, the sliding window examines every contiguous subarray of exactly that length exactly once. Adding the entering element and subtracting the leaving element maintains the correct window sum. The method checks the sum only when its length lies in [l, r] and the sum is greater than 0, so it considers every valid candidate.

Taking the minimum over all positive candidate sums gives the required result. If no candidate has a positive sum, answer remains Integer.MAX_VALUE and the method returns -1.

Complexity

Time: O(n * (r - l + 1)), which is O(n^2) in the worst case. Auxiliary space: O(1), since only the running window sum and answer are stored.

Implementation Notes

The LeetCode driver supplies nums as List<Integer>, so the method signature must be minimumSumSubarray(List<Integer> nums, int l, int r), not minimumSumSubarray(int[] nums, int l, int r). Use nums.get(index) to access elements and nums.size() for the array length.